

GISMA
University of Applied Sciences

M603 Advanced Algorithms

“The Campus Puzzle”
University Timetabling System
Project Report

Student Information

Full Name	Manoj Kumar
Student ID	GH1037914
Module	Advanced Algorithms M603A
Submission Date	18.09.2026

Project URL's

GitHub Repository	https://github.com/manojkumar-techdev/The-Campus-Puzzle
Video Demonstration	https://drive.google.com/file/d/1evywdyeTzoNeniSsaUe1ARTzOxSLlO40/view?usp=sharing

1. Executive Summary

This project uses a four-stage algorithmic pipeline in automated university timetabling. The system schedules the classes to the time slots and rooms, and achieves four hard constraints: room capacity, professor availability, student-group availability and room/slot-exclusivity. There are different techniques at each stage:

Stage	Algorithm	Role
1	Greedy	
2	Graph Colouring(Welsh-Powell)	Conflict Prevention
3	Dynamic Programming(bitmask)	Capacity-efficient room allocation
4	Recursive Backtracking	Best effort Gap filling

Final result: 19 classes were scheduled, 0 classes were in conflict, 190 seats were wasted and 1 class (HIST101 160 students) was not schedulable and was marked for manual scheduling.

2. Introduction

University timetabling is a problem of constraint satisfaction which involves graph colouring, bin packing and combinatorial optimisation. It is NP-hard in general, thus the use of a pipeline of complementary algorithms in practical systems.

The assessment brief needs to include the following: (1) a Greedy baseline; (2) a Welsh–Powell colouring of a conflict graph; (3) a room allocator for the DP which mixes recursive backtracking with best-effort; (4) a Conflict Report; (5) a Manual Fix Log.

3. Problem Definition

Classes (id, enrolment, professor, student groups), rooms (id, capacity), time slots (inputs (constraints.json))

Hard constraints:

#	Constraint	Description
C1	Capacity	Room capacity $\geq$ class enrolment
C2	Professor	No professor teaches two classes at the same time
C3	Student group	None of the students attend two classes at the same time
C4	Slot exclusivity	No two classes share a room at the same time during the day.

Each class uses one (slot, room) pair (C5)

Objectives (priorities): increase the number of classes that can be scheduled; reduce empty seats; avoid class conflicts; mark classes that can't be scheduled.

Test data: 20 classes, 8 rooms (30–150 seats), 6 slots, 1,360 students, 34 conflict edges.

4. System Architecture

/M603

```
|— main.py          # Pipeline orchestration
|— data/constraints.json  # Inputs
└— src/
    |— data_loader.py    # JSON loader
    |— schedule2.py      # Schedule / Assignment structures
    |— greedy_solver.py  # Stage 1
    |— graph_engine.py   # Stage 2
    |— optimizer.py      # Stage 3
    └— backtracker.py    # Stage 4
```

The **Schedule** object brings together validity checks, mutations, metrics and reporting. Each stage does not change previous stage results, only works on a shallow copy.

5. Stage 1 — Greedy Baseline

Algorithm: The classes sorted in the descending order by enrolment, where the enrolment is the number of students enrolled in that, class ID is used as it is the tie-breaker. The classes are also assigned in that where the first possible (slot, room) pair available that has enough seats. The baseline intentionally does not include professor and student group constraints, as they are naïve.

Justification: The "difficulty" measure is intrinsic to the enrollment size — the bigger the class the fewer possible rooms. Greedy is selected because it has polynomial time complexity.

Complexity: Time $O(C \log C + C \times S \times R)$; space $O(C + S \times R)$.

Result: 19 seats scheduled, 200 wasted, 8 student-group conflicts and 0.28 ms.

6. Stage 2 — The Graph Theory Conflict Engine.

Conflict graph: A student group or a professor will connect two nodes, which are corresponding two classes.

Welsh–Powell colouring: Vertices ordered by decreasing degree; each vertex gets the least used colour from adjacent vertices. No conflicts are present because adjacent nodes will always be in different colours.

Justification: The conflict structure can be directly modeled and is scalable to sparse graphs with graph colouring. Colouring does not allow conflicts, unlike Greedy which repairs them.

Complexity: Graph $O(C^2)$ worst case; colouring $O(V \log V + E \times \Delta)$; space $O(V + E)$.

Result: 20 nodes, 34 edges, 4 of 6 colours used, 20/20 classes coloured, 0 conflicts (down from 8), 0.25 ms.

7. Stage 3 — Dynamic Programming Room Allocator

Formulation Per slot:

$dp[i][mask]$ is min(waste) for assigning the first i classes with room subset mask $mask$

Base: $DP[0][0] = 0$

Rec: $DP[i][mask] = \min($

$DP[i-1][mask] + \text{SKIP_PENALTY}, \quad \# \text{ skip class } i$

$\min\{\text{free room } r \mid \text{such that it fits class } i:$

$DP[i-1][mask \cup \{r\}] + (\text{room}[r].\text{cap} - \text{class}[i].\text{enrolled})$

$)$

If DP has infeasible classes, it can defer them to Stage 4, but it does not require failure; this is provided by **SKIP_PENALTY**.

Why not brute force. Brute force tests all combinations of classes into rooms per slot — $O(R!)$. DP stores solutions to subproblems and reuses for optimal substructure and overlapping: each $(i, mask)$ state is solved only once.

Justification: The Fixed slots are also the exact and they have also the moderate number of the rooms present, so they are the feasible.

Complexity: The Time $O(S \times |C_t| \times 2^R \times R)$; space $O(S \times |C_t| \times 2^R)$.

Result: The scheduled are 19 and the wasted seats are from **200** → **190** (saving 10), 3.02 ms.

8. Stage 4 – Backtracking and Best-Effort Strategy

Mission: Classes may be re-assigned to different slots and tentative assignments can be undone in stage 4. Greedy doesn't allow any changes to the decisions and DP is per slot and doesn't allow re-assignment of classes placed.

Algorithm:

- Variable ordering (MCV + MRV): Variable ordering by conflict-graph degree descending, then fewest valid candidates, finally class ID.
- `_backtrack`: try each (slot, room) candidate for the first class; tentatively assign, forward-check the rest, recurse, unassign on failure.
- Forward checking: Once an assignment is made, check each of the remaining classes to see if there is at least one remaining candidate, if not, prune.
- Best-effort wrapper: make a full backtrack first, unschedule it if it is not found, put it in the wrapper without backward-checking the others if it is found, but backward-checking the others if it cannot be found.

Pruning strategy:

#	Pruning	Effect
2	Capacity pruning	Excludes rooms that are too small to be searched, before search
3	Laying branches to ground and mulching to prevent regrowth. ⁴	Deadheading.
3	MCV ordering	Surfaces dead ends early
4	MRV tiebreak	Fewest options first tiebreak: when two or more classes are the same degree, the one with the lowest number of options wins.

Value ordering: Candidates listed in the order of the smallest waste, then in the order of their index in the slot.

Complexity: Worst case $O((S \times R)^U)$. Our case: $U = 1, S = 6, R = 8 \rightarrow 48$ candidate checks.

Result: I'm still not sure if this is the most or the least number of scheduled days I've ever had for a job. 19/20 is optimal, and 0/0 is already optimal after Stage 3, 0.04 ms.

9. Conflict Report

- Scheduled CS101 11:00 R-101 Wasted 5 seats
- Scheduled MATH202 10:00 R-104 Wasted 2 seats
- Unscheduled HIST101 N/A N/A -- FLAG FOR MANUAL REVIEW

The student will recognize the distinct features of different historical eras. The student will identify the distinct characteristics of each historical period.

The last report will show all the scheduled classes with labels for each class (Perfect Fit or Wasted N seats) and clearly mark the single unscheduled class.

10. Manual Fix Log

UNSCHEDULED: HIST101 (prof P-13) 160 students

Reason: No room large enough (largest room = 150 seats)

MANUAL FIX LOG:

1. To Adding an evening slot (e.g. 17:00) and re-run main.py.
2. To Splitting the HIST101 into the two sections (e.g. 80 + 80).
3. Reserve an outside location that isn't already booked.

This converts a limitation in the algorithm into an administrative decision.

11. Pipeline Comparison

Metric	Greedy	Graph + DP	Final
Classes scheduled	19	19	19
Student-group conflicts	8	0	0
Total wasted seats	200	190	190
Runtime (ms)	0.28	3.27	3.31

Discussion: The largest of the qualitative gain was achieved by eliminating all 8 conflicts by the graph colouring method. DP saw a 10 % decrease in wasted seats, and no increase in conflicts. Backtracking has verified that the schedule is at maximum coverage: HIST101 is unschedulable, because there are 160 students and the largest room, 150 seats, is not enough — this was not a mathematical problem, it was an algorithmic problem.

12. Limitation and future work

Limitations: Backtracking is exponential worst-case, and is most appropriate for small unscheduled sets. The DP allocator makes the number of slots fixed. Soft constraints, such as preferences of the lecturer or equipment in the room, are not modelled.

Future Work: Local search or genetic algorithms for larger sets of unspecified problems; soft constraints as penalties; web interface; integration with external venues; multi-semester planning.

13. Conclusion

The result of this project was the complete, modular timetabling system that was built using the four complementary algorithmic stages. It was used to solve different sub-problems: Greedy – for speed; graph colouring – for conflict prevention; DP – for efficient use of exact capacity; and backtracking – for covering the maximum number of people with honest reporting.

The pipeline schedule has 19 classes scheduled, 20 classes conflict-free, 190 wasted seats and 1 class that is not yet scheduled, which is clearly marked as a manual intervention class. All the requirements of the M603 assessment brief have been met: Greedy baseline, Welsh–Powell colouring, DP allocator, recursive backtracking, best-effort strategy, Conflict Report, Manual Fix Log, complexity justification and pruning strategy.

References

1. Bellman, R. (1957). Dynamic Programming. Princeton University Press.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd Edition). MIT Press.
3. Gisma University of Applied Sciences. (2025). Advanced Algorithms Assessment Brief: M603.

4. Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
5. Welsh, D. J. A., & Powell, M. B. (1967). A maximum value of the chromatic number of a graph. *The Computer Journal*, 10(1), 85–86.
6. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
7. Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Pearson Education.
8. Skiena, S. S. (2020). *The Algorithm Design Manual* (3rd ed.). Springer.
9. Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.